

Transaction : It can be defined as set of tasks.

* A transaction is logically as set of related operations

* It can be defined as a logical unit of work on the data base. This may be an entire program, a piece of program or a single command (like SQL commands such as insert or delete) and it may engage in any no. of operations on the database.

Ex eg:-

A's Account:-

open Account(A)

old balance = A balance

new balance = old balance - 500

A balance = new balance

close Account(A)

B's Account:-

open Account(B)

old balance = B balance

new balance = old balance + 500

B balance = new balance

close Account(B)

* A transaction ensures that either all the operations are successfully executed (committed) or none of them taken effect (rolled back)

* Transactions are designed to maintain the data integrity, consistency and reliability of the database, even in the case of system failures or concurrent access.

Operations on Transactions:-

→ A user can make different types of requests to ACQU and modify the contents of a database.

Read operation:-

* A Read operation is used to Read the value of a particular database element and stores it in a temporary buffer in the main memory for further Actions such as displaying that value.

eg: For banking system, when a user checks their balance, a Read operation is performed on their account balance.

i.e: $\text{select balance from Accounts where Account_id = 'A123';}$

* This updates the balance of the user's Account after withdrawal.

Write operation:-

* It stores the updated data from main memory back to database. It usually follows a Read, where data is fetched, modified (e.g. Arithmetic changes) and then written back to save the updated value.

Subject: DBMS

Class Notes

Faculty: R.AKHILESH

Topic: Transaction:

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 02

eg: For banking system, if a user withdraws money, a write operation is performed after the balance is updated.

i.e: update account SET $\text{balance} = \text{balance} - 100$
where Account-id = 'A123'

* write the updated value of A from the buffer to the database.

for eg:

let say your account is A and your friend's account B, you are transferring 100 from A to B. The steps of the transactions are.

Before A: 500

B: 200

T1

T2

Read (A)

Read (B)

$A = A - 100$

$B = B + 100$

write (A)

write (B)

After A: 400

B: 300

3. commit: This operation in transaction is used to maintain integrity in the database. Due to some failure of power, hardware or software etc.

* A transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database.

i.e. might get inconsistent before and after transaction.

eg. After a successful money transfer in a banking system, a commit operation finalizes the transaction.

d.e commit;

* once the transaction is committed, the changes to the database are permanent and transaction is considered successful.

4. Roll back:-

A Rollback undoes all changes made by a transaction if an error occurs, restoring the database to its last consistent state.

* It helps to prevent data inconsistency and ensure safety.

eg. Suppose during money transfer process, the system encounters an issue like insufficient funds in the sender's account. In that case, the transaction is rolled back.

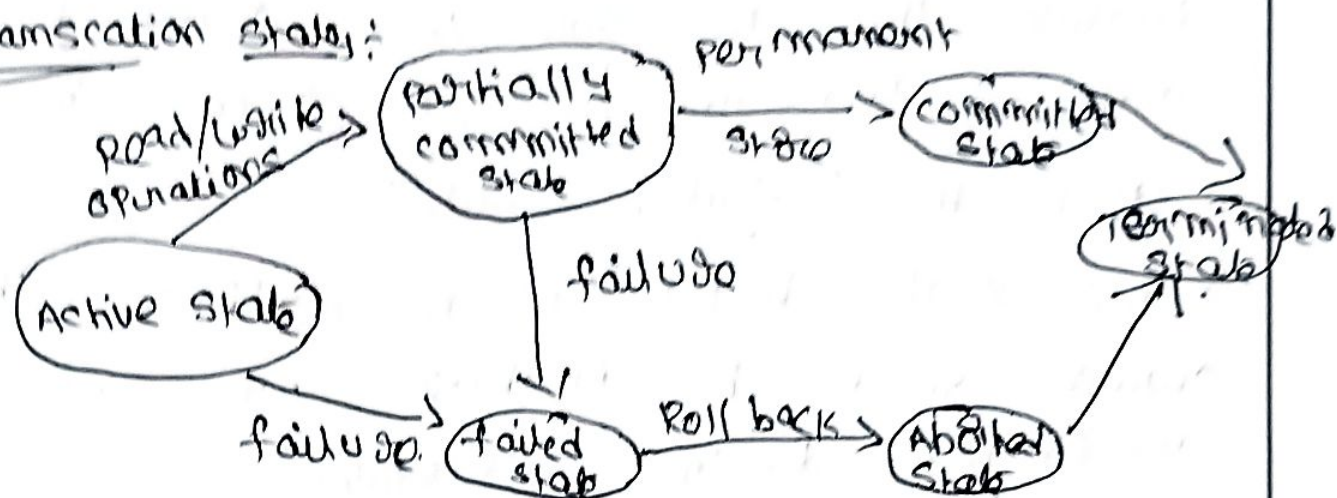
ROLLBACK;

⇒ This will undo all the operations performed so far and ensure the database remain consistent.

Transaction state represent the different phases of a transaction goes through during lifecycle in a database system.

* These state help track the progress and outcome of a transaction to ensure data integrity and consistency.

Transaction State:



Active state:

- * First state of transaction.
- * If a transaction is in execution, then it said to be in active state.
- * It does not matter which ~~the~~ step is in execution, until, unless the transaction is executing, it's remaining in Active state.
- * operation such as insertion, deletion or updation are performed during this state.

3. partially committed :-

- This state contains Read and write operations
- Reason why we have this state is because, a transaction can fail during execution.
- After completing all read and write operations the modifications are initially stored in main memory or local buffer.
- If the changes are made permanent in the database then the state will change to "committed state" and in case of failure it will go to the "failed state".

* This state helps us to rollback the changes made to database in case of a failure during execution.

3. Failed state :-

If a transaction is executing and a failure occurs either h/w or s/w failure then the transaction goes into failed state from the Active state.

* Here the database recovery system makes sure that the database is in a consistent state.

4. committed state :-

- If a transaction completes the execution successfully then all the changes made in

local memory during partially committed state are permanently stored in database.

* This marks the successful completion of a transaction.

5. Aborted State :-

* If a transaction fails during execution then the transaction goes into an failed state.

* The changes made into the local memory are rolled back to the previous consistent state and transaction goes into aborted state from the failed state.

* In Aborted state the DBMS Recovery system performs one of two actions

1. Kill the transaction :- The system terminates the transaction to prevent it from affecting other operations

2. Restart the transaction :- After making necessary adjustments, the system reverts the transaction to an Active state and attempts to continue its execution.

* It may occur due to h/w or software failure then the transaction can be restarted and considered as new transaction

6. Terminated State:-

- This is last state in the life cycle of transaction.
- After entering the committed state or aborted state, then the transaction finally enters into terminated state.
- It refers to the final state of a transaction indicating that it has completed its execution.
- In this state, no further actions are permitted from the transaction, as the database is now stable.

for eg: user wants to transfer \$500 from Account A to Account B, the system should handle the following transaction states.

Active state: Read balance of Account A = \$1000

partially committed state: Deduct \$500 Account A's balance ($\$1000 - \$500 = \$500$) and temporarily update Account B's balance (add \$500)

committed state: Account A's new balance = \$500, Account B's new balance = \$1500 (changes are written to database)

Failed state: system crashes after deducting \$500 from Account A but before adding to Account B.

Aborted state: Account A's balance restored to \$1000, no changes are made to B's Account

Subject: DBMS

Faculty: R. Akhilesh Reddy

Topic: ACID Properties:

Class Notes

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 05

* Transactions are fundamental operations that allow us to modify and retrieve data.

→ How ever to ensure that integrity of a database, it is important that these transactions are executed in a way that maintains consistency, robustness and reliability even in the case of failure/errors.

ACID

A-Atomicity

All or

nothing

transactions

C-consistency

valid state

maintenance

I-isolation

Transactions

do not

affect

each other

D=Durability

Permanent

changes

After

commitment.

Atomicity:-

* It states that all operations of transactions take place at once, if not the transactions is aborted.

* There is no midway, i.e the transaction can not occur partially. Each transaction is treated as one unit and either Run to completion or is not executed at all.

* Atomicity means a transaction is all-or-nothing either all its operations are succeed or none are applied.

eg:- consider the following Transaction T consisting of $T1$ and $T2$: Transfer of \$100 from Account X to Account Y .

Before $X: 500$ $Y: 200$	
Transaction T	
$T1$	$T2$
$Read(X)$ $X := X - 100$ $Write(X)$	$Read(Y)$ $Y := Y + 100$ $Write(Y)$
After: $X = 400$	$Y = 300$

* If transaction fails After completion of $T1$ but before completion of $T2$, the database would be left in inconsistent state.

* with Atomicity, if any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

consistency :-

* The transactions remain in valid state before and after a transaction.

* The integrity constraints are maintained so that the database is consistent before and after transaction.

* A valid state follows all defined Rules, constraints and Relationships (like primary keys and foreign keys)

* If transaction violates any of these rules, it is rolled back to prevent corrupt or invalid data.

→ The consistent property of database states that every transaction sees a consistent database instance.

→ The transaction is used to transform the database from one consistent state to another consistent state.

EG: Suppose the sum of all balances in a bank system should always be constant.

* Before a transfer, the total balance is \$700. After the transaction, total balance should remain \$700.

* If the transaction fails in the middle (like updating one account but not other) the system should maintain its consistency by rolling back the transaction.

Total before T occurs = $500 + 200 = 700$

Total After T occurs = $400 + 300 = 700$

X = 500

T

Read (X)

$X := X * 100$

Write (X)

Read (Y)

$Y := Y * 50$

Write (Y)

Y = 500

T

Read (X)

Read (Y)

$Z := X + Y$

Write (Z)

Isolation:

- * It shows that the data which is used at the time of execution of transaction cannot be used by second transaction until first one is completed.
- It ensures that transactions run independently without affecting each other.
- Changes made by one transaction are not visible to others until they are committed.
- * It ensures that the result of concurrent transaction is the same as if they were run one after another, preventing issues like:
 - Dirty Reads: Reading un-committed data.
 - Non-repeatable Reads: data change b/w two reads.
 - Phantom Reads: new rows appear during a transaction.
- * The concurrency control subsystem of the DBMS enforces the isolation property.
- * In isolation, if the transaction T_1 is being executed and using data item x , then the data item can't be accessed by any other transaction T_2 until the transaction T_1 ends.

eg:- $x = 500$ T

Read (x)

 $x = x - 50$

write (x)

Read (y)

 $y = y + 50$

write (y)

 $y = 500$ T''

Read (x)

Read (y)

 $z = x + y$

write (z)

* consider two transaction T and T''

 $x = 500, y = 500$ Explanation:-

1. Transaction T:

→ T wants to transfer \$50 from x to y

→ T reads (y value: 500): deducts \$50 from

x (new $x = 450$) and add \$50 to y(new $y = 550$).

2. Transaction T'':

→ T'' starts reads ($x: 500$) and ($y: 500$)→ It calculates the sum: $500 + 500 = 1000$

→ Meanwhile, values of x and y change

to 450 and 550. ($450 + 550 = 1000$)

→ Isolation ensures that T'' does not read outdated values while another Transaction (T) still in progress.

* Transaction should be independent and T^* should
Access the final values only after T commits.

4. Durability :

→ It is used to indicate the performance of
the database consistent state.

→ It states that the transaction made the
permanent changes.

* They cannot be lost by the error operation
of a faulty transactions or by system failure.

* When a transaction is completed, then the
database reaches a state is known as the
consistent state; that consistent state
can not be lost, even in the event of
system failure.

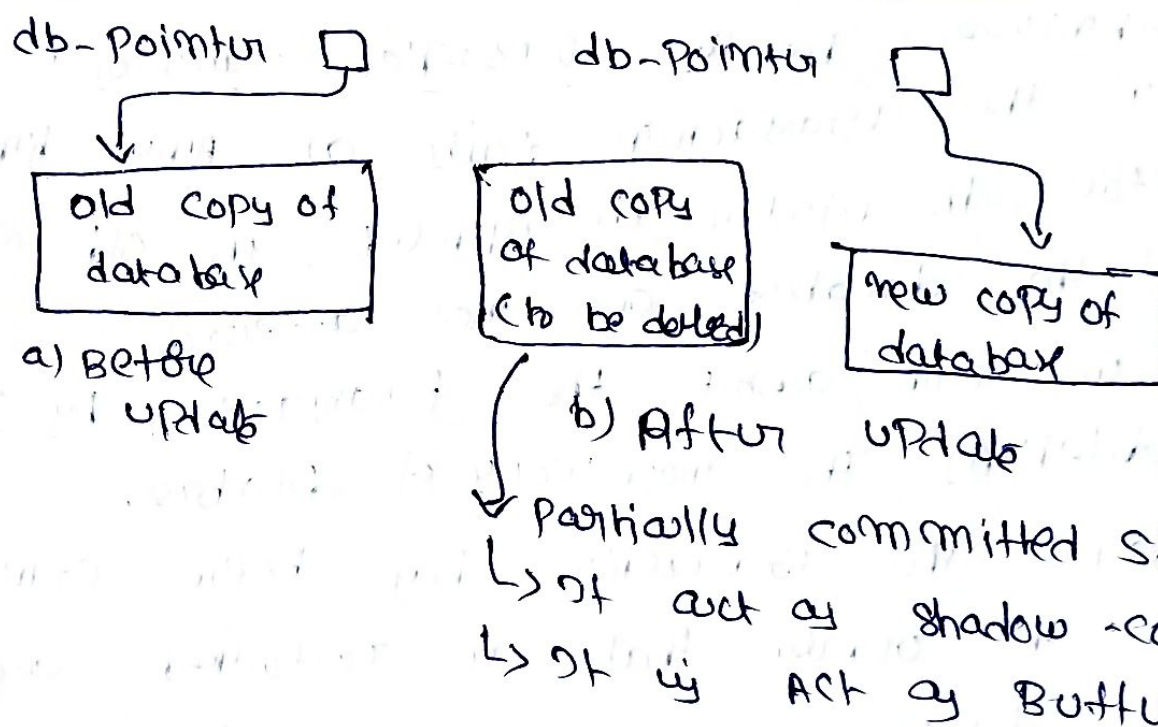
* The recovery subsystem of DBMS has
the responsibility of durability property.

Ex: After successfully transferring money
from Account A to Account B, the changes
are stored in disk.

* Even if there is a crash immediately
after the commit, the transfer details
will still be intact when system recovers
ensuring durability.

- * The Recovery management component of a database system implement the support for Atomicity & durability.
- * we will use shadow-database scheme to implement Atomicity & durability, it is also called as shadow-copy technique.
- * There are many techniques that are employed by recovery management component for ensuring Atomicity & durability.
- * In this technique as 'shadow-copy', duplicate copies of database called shadow copies are created.

Working of shadow-copy Technique



→ In the shadow-copy technique, a transaction that wants to update the database first creates a complete copy of database.

→ All updates are done on the new-database copy, leaving the original copy, the shadow copy.

→ If at any point of transaction has to be aborted, the system merely deletes new copy, and old copy of the database has not been affected.

→ This scheme is based on making copies of the database, called shadow copies assumes that only one transaction is active at a time.

→ A pointer called db-pointer is maintained on disk, it points to the current copy of the database.

Technique handling transaction failure:

* If the transaction fails at any time before db-pointer is updated, the old contents of the database are not affected.

* We can abort the transaction by just deleting the new copy of database.

* Once the transaction has been committed, all the updates that is performed are in database pointed by db-pointer.

Subject: DBMS

Faculty: R. Akhilesh Reddy

Topic: concurrent

Class Notes

Execution:

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No.... of SP

Book Reference:

Date Conducted:

Page No: 02

for eg:

T1

T1

Read (sam)

Sam = Sam - 200

write (sam)

Read (sam)

Sam = Sam - (Sam * 20/100)

write (sam)

Read (john)

john = john + 200

write (john)

Read (john)

john = john + (Sam * 20/100)

write (john)

Problems with concurrent execution:

→ In a database transaction, the two main operations are Read & write, so there is a need to manage these two operations efficiently, otherwise it may lead to inconsistency.

Problem 1: Lost update problem

(w-w conflict)

<u>Time</u>	<u>T_x</u>	<u>T_y</u>
t_1	Read(A)	—
t_2	$A = A - 50$	—
t_3	—	Read(A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	write(A)	—
t_7	—	write(A)

→ At time t_1 , transaction T_x reads the value of Account A i.e. \$300

→ At time t_2 , transaction T_x deducts \$50 from Account (A) that becomes \$250 (only deducted and not updated/write)

→ At time t_3 , T_y reads the value of Account A (i.e. \$300) only because T_x does not update value yet

→ At time t_4 , T_y adds \$100 to Account A that becomes \$400 (only added, not updated/write)

→ At time t_6 , T_x writes the value of Account A (i.e. \$250), T_y does not update value yet

→ At time t_7 , T_y writes the value of Account A (i.e. \$400). T_x is lost,

Def: In a multi-user system, multiple users can access and use the same database at one time is known as concurrent execution of the database.

* It means that the same database is executed ~~simultaneously~~ for multiple transactions concurrently.

* It is similar to ~~multi~~ multi-programming approach performed in operating system.

for eg:

In multi-user system multiple users can access and use the same database at one time, is called concurrent executions.

* It means that same database is executed - simultaneously on a multi-user system by different users.

* It is useful for following reasons.

→ Increase the efficiency / throughput of a system

→ Decreases the response time and Average time

Q8 eg: Serial Execution ($T1 \rightarrow T2$)

↳
first

$T1$

$T2$

Read (sam)

Sam = Sam - 200

Write (sam)

Read (john)

john = john + 200

Write (john)

Read (sam)

Sam = Sam - (Sam * 20/100)

Write (Sam)

Read (john)

john = john + (Sam * 20/100)

Write (john)

→ for eg: $T2 \rightarrow T1$, $T2 \rightarrow$ executed first then $T1$ executes.

→ The whole transaction performed at a time is called serial transaction.

* The thing is that the simultaneous execution is performed should be done in an interleaved manner and no operation should affect the other executing operations to maintain consistency of the database.

Subject: DBMS
Faculty: R. AKHILESH
Topic: Schedule

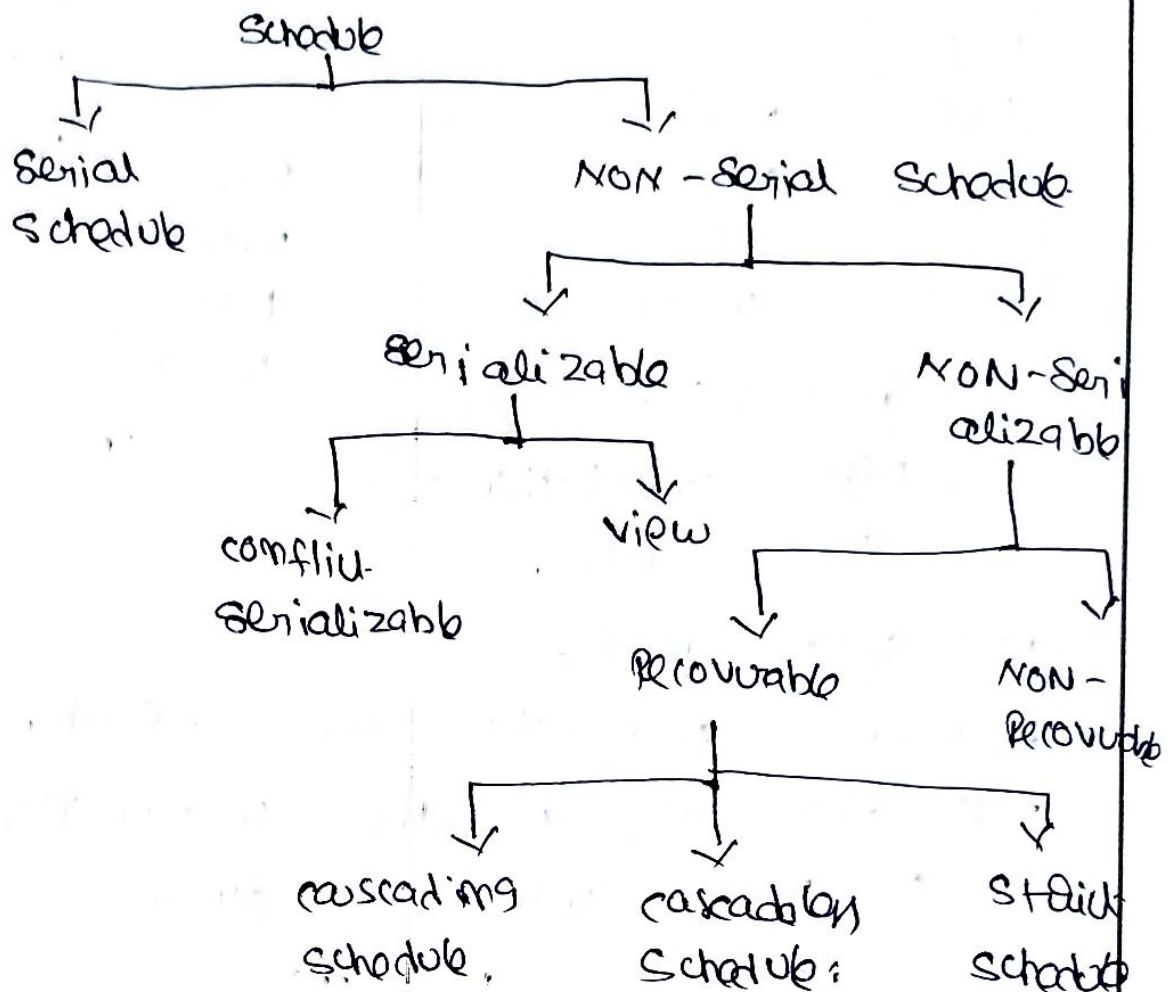
Class Notes

Unit No: 5
Lecture No:
Link to Session
Planner (SP): S.No.... of SP
Book Reference:
Date Conducted:
Page No: 11

* A series of operations from one transaction to another transaction is known as schedule.

* It is used to preserve the order of operation in each of individual transaction.

* It is divided into diff - type



Serial Schedule:

→ All the transactions executes serially one after the other.

→ when one transaction executes, no other transaction is allowed to execute.

T1
Read(A)

T2
Read(A)

T3
Read(A)
write

T1 → T2 → T3
→ one After Another.

eg:-

T1
R(x)
W(x)
R(y)
W(y)

T2
R(x)
W(x)
R(y)
W(y)

T1

R(x)
W(x)
R(y)
W(y)

T2
R(x)
W(x)
R(y)
W(y)

T1 → T2 : T1 started
After completion only
T2 starts

T2 → T1

NON-Serial schedule:-

- multiple transactions are executed concurrently.
- operations of all the transaction interleaved or mixed with each other.

T1
R(A)
W(B)
R(B)
W(B)
commit

T2
R(A)
R(B)
commit

Subject: DBMS

Class Notes

Faculty: P. Akhilesh

Topic: Serializability

Unit No: Class

Lecture No:

Link to Session

Planner (SP): S.No. ... of SP

Book Reference:

Date Conducted:

Page No: 12

Def: A serializable schedule always leave the database in consistent state.

* on the concept in a transaction that helps to identify which non-serial schedule is correct and will maintain the database consistency.

→ It relates to the isolation property of transaction in the database.

* serializability is the concurrency scheme where where the execution of concurrent transactions is equivalent to the transactions which execute serially.

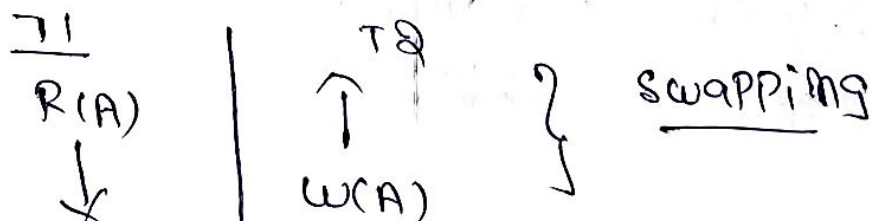
Types of Serializability:

→ conflict serializability

→ view serializability

conflict serializability: it can be used to check whether a non-serial schedule is conflict serializable or not

* A schedule is called conflict serializable, if we can convert it into a serial schedule after swapping its non-conflicting operations



conflict operations:-

R-W

W-R

W-W

T1	T2
R(A)	W(A)

→ At T1 → it performs Read operation.
 T2 → performs write operation
 on same time (i.e. R-W conflict)
 working on same data item.

→

T1	T2
W(A)	R(A)

T1 → write }
 T2 → Read } w-R
 conflict
 At sometime.

→

T1	T2
W(A)	W(A)

T1 → write }
 T2 → write } w-w
 conflict
 At sometime.

* Two operations are said to be conflict, if they satisfy all the following three conditions.

→ Both operations should belong to different transactions.

→ Both operations are working on same data item.

→ At least one of operation is a write operation.

Subject: DBMS
 Faculty: R. Ashish
 Topic: conflict Serializability

Class Notes

Subject

Unit No: _____
 Lecture No: _____
 Link to Session
 Planner (SP): S.No. of SP
 Book Reference: _____
 Date Conducted: _____
 Page No: 13

eg: operation $w(x)$ of transaction T_1 and operation $R(x)$ of transaction T_2 are conflicting operations, because it satisfy all three conditions mentioned above.

T_1	T_2
$w(x)$	$R(x)$

eg 2: Similarly operations $w(x)$ of T_1 and $w(x)$ of T_2 are conflicting operations

T_1	T_2
$w(x)$	$w(x)$

} satisfy All three conditions

eg 3: operations of $w(x)$ of T_1 and $w(y)$ of T_2 are non-conflicting operations.

T_1	T_2
$w(x)$ ↓	$w(y)$ ↑

} non-conflicting operations

→ Swapping can be done, changing the positions to convert to serial schedule

eg 4: Similarly $R(x)$ of T_1 and $R(x)$ of T_2 are non-conflicting operations because none of them is write operation

T_1	T_2
$R(x)$	$R(x)$

} non-conflicting

Q9:-

T1	T2
R(A)	
W(A)	
↑	
R(B)	R(A)
	↓

→ concurrent:

conflict

$W(A) - R(A)$

non-conflict

$R(B) - R(A)$

~~R(B)~~

To make above concurrent transaction into serial we swapped non-conflicting operations i.e. $R(A)$ in T2 and $R(B)$ in T1.

T1	T2
R(A)	
W(A)	
R(B)	
↓	
	R(A)

Serializable.

Q9:-

T1	T2
R(A)	
	R(A)
	R(B)
	W(B)
R(B)	
W(A)	

conflict:

X

non-conflict

$R(A) - R(A)$

$R(A) - R(B)$

$R(A) - W(B)$

→ serial schedul.

1)

T1	T2
	R(A)
R(A)	R(B)
	W(B)
R(B)	
W(A)	

2)

T1	T2
	R(A)
R(A)	R(B)
	W(B)
R(B)	
W(A)	

3)

T1	T2
	R(A)
	R(B)
	W(B)
R(A)	
R(B)	
W(A)	

Subject: DBMS

Class Notes

Faculty: R. Akhilesh

Topic: View Serializability

Unit No: Class

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 14

* It is a process to find out the given schedule is view serializable or not.

→ To check whether a given schedule is view serializable, we need to check whether the given schedule is view equivalent to its serial schedule.

Serial schedule:
→ If a schedule is conflict serializable, if it is view equivalent to a serial schedule.

→ The view serializable which does not contain conflict serializable contains blind writing.

→ Performing writing operation without reading operation such write operation is called blind write.

View Equivalent:-

Two schedules S1 and S2 are said to be view equivalent if they satisfy all the following conditions.

* Initial Read.

* Final Write

* Update Read.

* Initial Read :- Initial Read of each data item in transaction must match in both.

schedules.

* for example, if transaction T_1 reads a data item x before transaction T_2 in schedule S_1 then in schedule S_2 , T_1 should be read x before T_2 .

* if a transaction T_1 is reading the data item A , then in S_2 , transaction T_1 should also read (A) .

T_1	T_2
Read(A)	Write(A)

T_1	T_2
Read(A)	Write(A)

final write : final write operations on each data item must match in both of the schedules.

for eg: a data item x is last written by transaction T_1 in S_1 then in S_2 , the last write operation on x should be performed by T_1 .

T_1	T_2	T_3
w(A)	R(A)	w(A)

Schedule

S_1

T_1	T_2	T_3
w(A)	R(A)	w(A)

Schedule (S_2)

Subject: DBMS

Faculty: R. Akhilesh

Topic: Serializability.

Class Notes

Subject:

Faculty:

Topic:

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 15

Update

Final Read: If in schedule S1, then
Transaction T1 is reading data item

updated by T2 then in S2, T1 should be
read the value after the write operation
of T2 on same data item.

for eg: In S1 T1 $\rightarrow$ reads on X after
the write operation on X by T2.
then in S2, T1 should read X.
After T2 performs write on X.

T1	T2	T3
1. w(A)	2. w(A)	3. R(A)

S1

T1	T2	T3
2. w(A)	1. w(A)	3. Read(A)

S2

* Above two schedules are not view equal
because in S1, T3 is read (A) updating
by T2 and in S2, T3 is reading
A updated by T1

S1

T1 - w(A)

T2 - w(A)

T3 - Reading A

Updated by T2

S2

T2 - w(A)

T1 - w(A)

T3 - Reading A

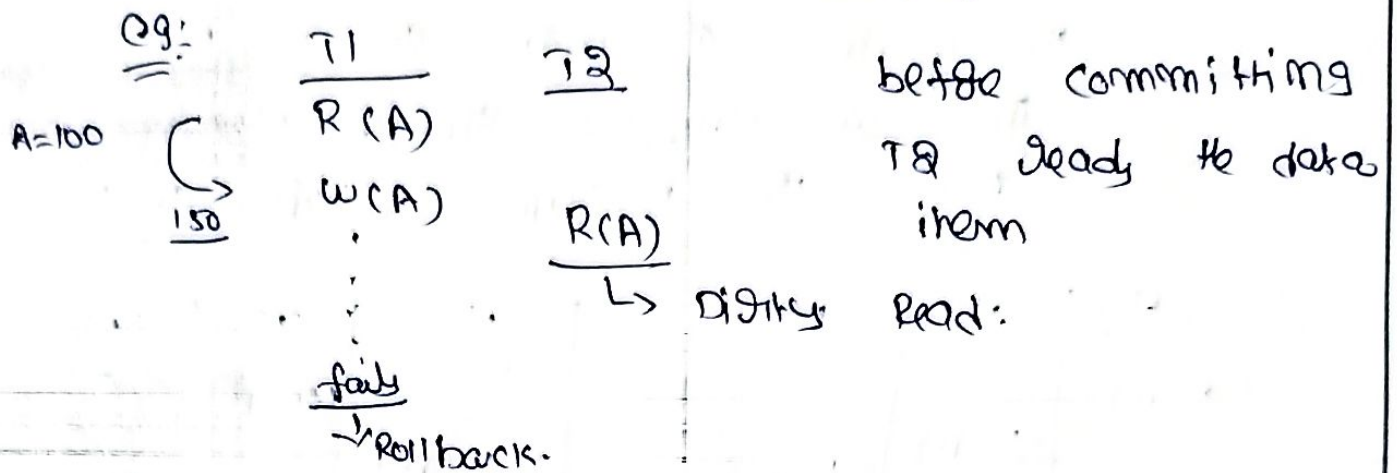
Updated by T1

$\rightarrow$ not equal

Recoverability :-

If in a schedule.

1. A transaction performs a dirty read operation from an un-committed transaction.
2. And its commit operation is delayed till the un-committed transaction either commits or rolls back then such a schedule is known as a Recoverable Schedule.



Here

* The commit operation of the transaction that performs the dirty read is delayed.

* This ensures that it still has a chance to recover if the un-committed transaction fails later.

T1	T2
R(A)	
W(A)	
⋮	
⋮	R(A) // Dirty Read.
⋮	W(A)
commit.	commit // Delayed

Recoverable Schedule.

Subject: DBMS

Faculty: R. Akhila

Topic: Recoverability

Class Notes

Unit No: Class

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 16

Non-Recoverable Schedule:

→ If in a schedule, a transaction performs a dirty read operation from an un-committed transaction and commits before the transaction from which it has read the value then it is called Non-Recoverable schedule.

Ex:

A = 100

A = 150

T1	T2
R(A)	
W(A)	
⋮	
⋮	
⋮	
Roll back	

R(A) // dirty read 150
W(A) 200
commit

* Recoverable schedule is divided into three types.

1. Cascading schedule:

If in a schedule, failure of one transaction causes several other dependent transactions to roll back or abort is called a cascading schedule.

→ It simply leads to wastage of CPU time.

Subject: DBMS
Faculty: R. Akhilesh

Class Notes

Topic: Recoverability:

strict schedule:

→ If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted is called strict schedule.

→ It allows only committed read and write operations.

eg:-

T1	T2
R(A)	
W(A)	
commit	R(A)
	W(A)
	commit

eg:-

T1	T2
R(A)	
W(A)	R(B)
	W(B)
commit	
	R(A)
	commit

Unit No: _____
Lecture No: _____
Link to Session _____
Planner (SP): S.No. of SP
Book Reference: _____
Date Conducted: _____
Page No: 17

Isolation :- It ensuring that the operations of one transaction remains hidden from other transaction until completion.

* It means that no two transaction should interfere with each other and affect other's intermediate state.

Phenomena defining Isolation:-

→ Dirty Read :- A Dirty Read is the situation when a transaction reads a data that has not yet been committed.

→ Non-Repeatable Read :- when a transaction reads same row twice and get a different value each time.

→ Phantom Read :- when two same queries are executed, but the rows retrieved by two are different.

Level of Isolation:-

→ Serializable :- This is the highest isolation level, A Serializable execution is guaranteed to be serializable.

* Serializable execution is defined to be an execution of operations in which concurrently executing appears to be serially executing.

Subject: DBMS

Class Notes

Faculty: R. Akhila

Topic:

Isolation & Testing for Serializability

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No. ... of SP

Book Reference:

Date Conducted:

Page No: 18

2. Repeatable Read - This is the most restrictive isolation level. This allows transactions to be accepted after they have begun, even if they have not completed.

* This level enables Phantom Reads.

3. Read un-committed :- In this level, one transaction may read not yet committed changes made by other transactions, thereby allowing dirty reads.

* These transactions are not isolated from each other.

4. Read - committed :- This isolation level guarantees that any data read is committed at the moment it is read.

-> It does not allow dirty reads.

-> The transaction holds a read or write lock on the current row.

Testing for Serializability :-

-> Serialization graph is used to test the serializability of a schedule.

-> Assume a schedule S, for S we construct a graph known as precedence graph.

* This graph has a pair $U = (V, E)$

→ The set of vertices V is used to contain all the transactions participating in the schedule.

→ The set of edges E is used to contain all the edges $T_i \rightarrow T_j$ for which one of these conditions holds.

1. create an edge $T_i \rightarrow T_j$, if T_i executes write(a) before executes read(a)

2. create an edge $T_i \rightarrow T_j$, if T_i executes read(a) before executes write(a)

3. create an edge $T_i \rightarrow T_j$ if T_i executes write(a) before T_j executes write(a).

* Precedence graph for schedule is

$T_i \rightarrow T_j$

R-W

W-R

W-W

* If a precedence graph contains a single edge $T_i \rightarrow T_j$ then all the instructions of T_i are executed before the first instruction of T_j is executed.

→ If graph for schedule S contains a cycle. Then S is non-serializable, If no cycle then S is known as serializable

→ Implementing Isolation typically involves concurrency control mechanisms.

Lock-Based Protocols:-

* concurrency control technique implements some protocols.

* LOCK BASED

* TIME STAMP

* Validation Based

* Multiple GRANULARITY

Lock-Based Protocols:-

In this type of protocol, Any transaction cannot read or write data until it acquires an appropriate lock on it.

* It is divided into two-type:

→ Shared Lock

→ Exclusive Lock.

Shared Lock:-

* It is also known as Read-only Lock, In a shared lock, the data item can only read by the transaction.

* It can be shared b/w transactions because when the transaction holds a lock, then it can't update the data on the data item.

* It is denoted by S(A)

T ₁	T ₂
S(A)	S(A)
R(A)	R(A)

* same data item can be locked,

Exclusive lock:-

- The data item can be both read as well as written by transaction.
- This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously.

* It is denoted by $X(A)$

T1	T2
$X(A)$	
$R(A)$	
$W(A)$	$\leftarrow X$

Compatibility Table:-

- * There ~~will~~ will be multiple transactions that requests to acquire locks for respective data items in which consistency component checks compatibility table to grant the locks.

Request	T1	T2
S	$\bullet$	X
X	S	X
X	X	X

T1	T2
$S(A)$	
$R(A)$	
$W(A)$	
	$X(A)$
	$R(A)$
	$W(A)$
	$U(A)$

- unlock(A):- After performing the particular operation on transaction, need to be un-locking then only the transaction will perform required operations.

Two-phase Locking Protocol:

* Also known as 2PL Protocol

* It divides the execution phase of transaction into two parts:

Step-1: In the first part, when execution of transaction, it seeks permission for lock it requires

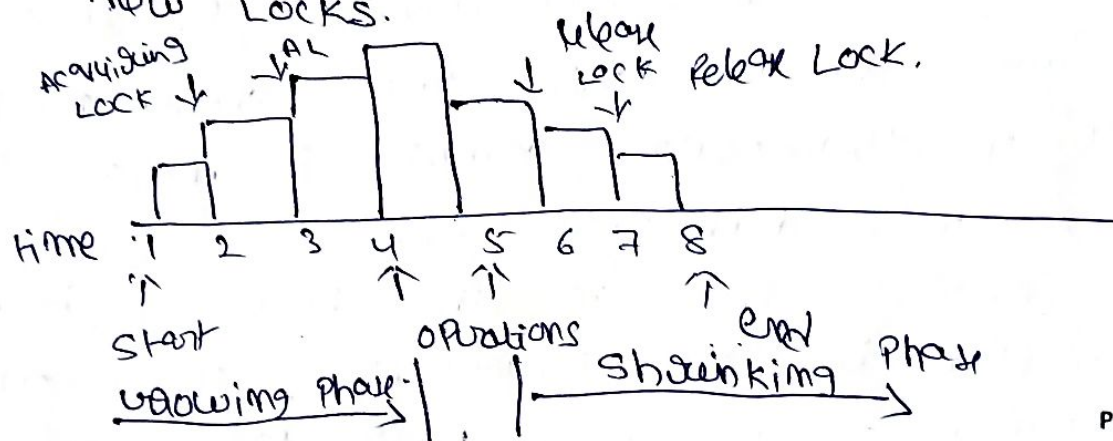
1:- In the second part, the transaction acquires all the locks.

2:- In this phase, transaction can not demand any new locks, it only release the acquired locks.

* The 2PL protocol allows each transaction to make a lock or unlock request in two steps:

Growing Phase: In this phase transaction may obtain locks, but not release any locks.

Shrinking Phase: In this phase, a transaction may release locks but not obtain any new locks.

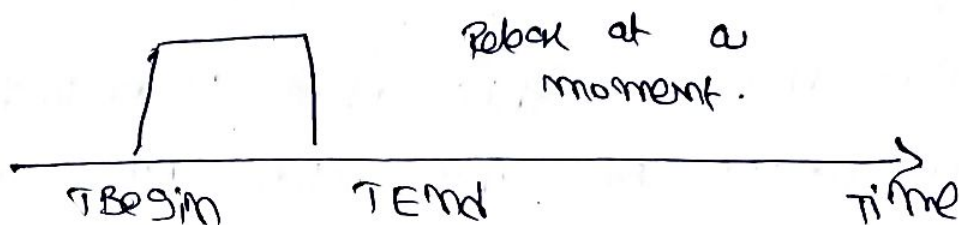


TYPES of 2PL Protocol:

1. Strict Two-Phase Locking Protocol
2. Rigorous " " "
3. conservative " " "

Strict Two-PL Protocol:

- It is similar to 2PL, in the first acquiring all the locks, the transaction continues to execute normally.
- The only diff b/w 2PL & Strict 2PL does not release a lock after using it.
- Strict 2PL waits until the whole transaction to commit and then it releases the all locks at a time.
- It does not have shrinking phase of lock release.



- * It avoids cascaded roll back.
- * This protocol not only provides two-phase locking but also all exclusive locks should be held until the transaction commits or aborts.
- * It is not dead-lock free.
- * It ensures that if the data is being modified by one transaction, then other transaction can not read until first transaction commit.

Q.NO

Rigorous 2PL:

→ A transaction can acquire locks on data item when ever it requires (only in growing phase) during its execution

* It has growing phase, It does not have a shrinking phase.

* It does not ensure Dead-LOCK-free schedul.

* In Rigorous 2-PL a transaction only reads value of committed transaction.

conservative 2PL:

→ A transaction has to acquire locks on all the data items it requires before the transaction begins its execution

* It does not have a growing phase.

* It ensures the schedule generated would be serializable and Dead-LOCK free.

* In conservative 2-PL a transaction can read a value of un-committed transaction.

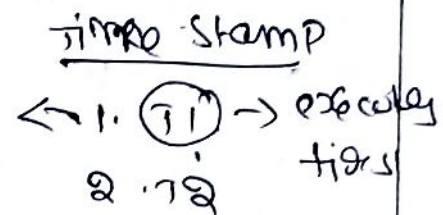
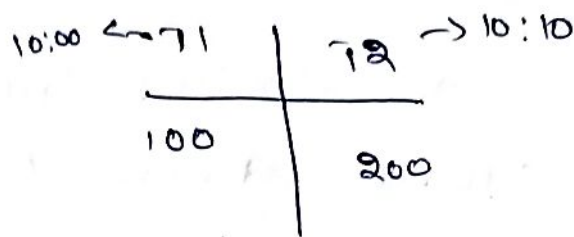
Time Stamp Protocol

→ The time stamp ordering protocol is used to order the transaction based on their time stamps. The order of transaction is nothing but the ascending order of the transaction creation.

* The priority of the older transaction is higher, that's why it executes,

* To determine the time stamp of the transaction this protocol uses system time & logical counter.

* The lock-based protocol is used to manage the order b/w conflicting pairs among transactions at execution time, but time-stamp based protocols starts working as soon as transaction is created.



eg: let's assume there are two transactions T1 & T2. Suppose T1 has entered system at 007 time and T2 has entered 009 time. Then T1 has the highest priority.

Time stamp: unique value assigned to every transaction.

Working:

1. when a transaction T_i publishes Read(x) operation, then following conditions are checked.

a) If $W-TS(x) > TS(T_i)$ operation gets rejected T_i is rolled back.

W-TS $\rightarrow$ write time stamp

TS(T_i) $\rightarrow$ time stamp of transaction (T)

R-TS $\rightarrow$ read time stamp.

b) If $W-TS(x) \leq TS(T_i)$ read operation is executed
~~set~~ $R-TS(x) = \max \{ R-TS(x), TS(T_i) \}$

When a transaction T_i publishes write(x) operation, the following conditions are checked.

a) If $R-TS(x) > TS(T_i)$ operation gets rejected and T_i is rolled back.

b) If $W-TS(x) > TS(T_i)$ the operation is gets rejected and T_i is rolled back.

c) otherwise execute the write operation

set $W-TS(x) = TS(T_i)$

for eg:

100	200
T_1	T_2
R(A)	
	W(A)

T_1	T_2
W(A)	
	R(A)

T_1	T_2
W(A)	
	W(A)

T_1	T_2
	R(A)
W(A)	
200	
commit	
	fail

Roll back $\rightarrow$

T_1	T_2
W(A)	
A=200	
commit	
	W(A)
	A=100

consider:-

$$TS(T1) = 100 \quad TS(T2) = 200 \quad TS(T3) = 300.$$

T1	T2	T3
R(A)	R(B)	
W(C)	W(B)	R(B)
		W(A)

	A	B	C	300
RTS	0	0	0	
WTS	0	0	100	
	1			300

T1 - R(A):

Step 1: $WTS(A) > TS(T1)$

$$0 > 100 \quad \times$$

Set $RTS(A) = \max \{0, 100\} \rightarrow \max \{RTS(A), TS(T1)\}$
 $= 100 \rightarrow \max \{0, 100\}$

T2 - R(B):

Step 1: $WTS(B) > TS(T2)$

$$0 > 200 \quad \times \rightarrow \text{condition fails}$$

Set $RTS(B) = \max \{0, 200\} = 200.$

T1 - W(C):

Step 2: a) $RTS(C) > TS(T1)$

$$0 > 100 \rightarrow \text{condition fails}$$

b) $WTS(C) > TS(T1)$

$$0 > 100 \rightarrow \text{condition false}$$

execute $W(C)$

Set $WTS(C) = TS(T1)$
 $= 100$

T3 - R(B)

Step 1: $WTS(B) > TS(T3)$

$$0 > 300$$

$$\text{Set } RTS(B) = \max\{200, 300\} = \underline{300}$$

T2 - W(B)

Step 2: a) $RTS(B) > TS(T2)$

$300 > 200 \rightarrow$ condition true
operation get Rejected, Roll back.

T3 - W(A)

Step 2: $RTS(A) > TS(T3)$

$$100 > 300 \rightarrow \text{condition false.}$$

$$0 > 300 \rightarrow \text{condition false.}$$

$$\text{Set } WTS(A) = \max\{100, 300\} = 300$$

Validation Based Protocol:

$\rightarrow$ It is also known as optimistic Protocol

$\rightarrow$ It is method to avoid concurrency in transaction. In this Protocol, the Local copies of transaction data are updated rather than database itself, which results in less interference while execution of transaction.

$\rightarrow$ It contains 3-Phase.

Read: In this phase, the data values from the database can be read by a transaction but write operation or updates are only applied to the local data copy not the Actual database.

Validation Phase:-

→ In validation phase, the data is checked to ensure that there is no violation of serializability while applying the transaction updates to the database.

Write Phase:-

→ In the write phase, the updates are applied to the database if the validation is successful else, the updates are not applied and the transaction is rolled back.

→ Here for each phase the following diff time stamps

* start (T_i) :- gives the time when T_i starts execution

* validation (T_i) :- gives the time when T_i finishes its read phase and starts validation phase.

* finish (T_i) :- gives the time when T_i is finished its execution or write phase.

→ To clear the all the validation test by T_i , then must satisfy one of the following condition.

i) $\text{finish}(T_i) < \text{start}(T_j)$:- It means T_i is older than and its finished before T_j starts.

ii) $\text{finish}(T_i) < \text{validate}(T_j)$:- Ensures Actual write by T_i & T_j will not overlap

iii) $\text{validate}(T_i) < \text{validate}(T_j)$:- Ensures the T_i has complete read phase before T_j completes.

Subject: DBMS
Faculty: R. Akhilesh
Topic:

Class Notes

Unit No: _____
Lecture No: _____
Link to Session
Planner (SP): S.No. _____ of SP
Book Reference: _____
Date Conducted: _____
Page No: 28

Validation Based Protocol:-

for eg:-

T_i

S	V	F
5	10	15

T_j

S	V	F
19	29	39

1. $\text{finish}(T_i) < \text{start}(T_j)$

$15 < 19 \quad \checkmark \quad (\text{No overlap})$

2. $\text{finish}(T_i) < \text{validation}(T_j)$

$15 < 29 \quad \checkmark \quad (\text{No overlap})$

3. $\text{validate}(T_i) < \text{validate}(T_j)$

$10 < 29 \quad \checkmark \quad (\text{No overlap})$

Multiple Granularity:-

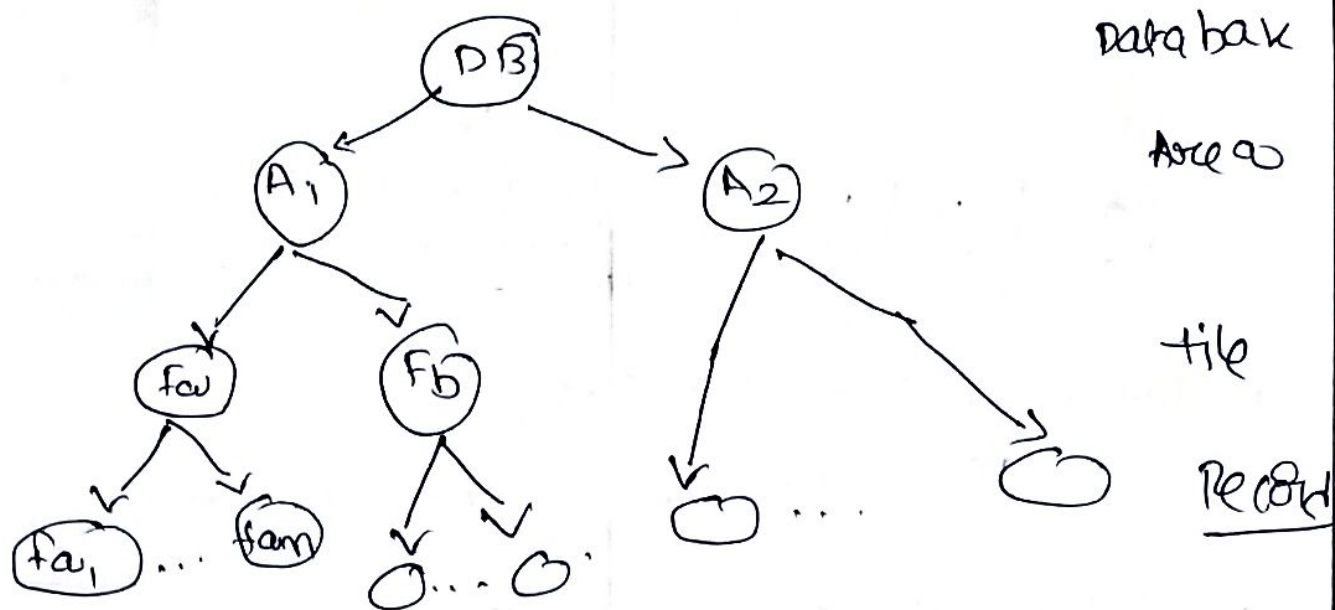
→ It can be defined as hierarchically breaking up the database into blocks which can be locked

→ It enhances concurrency & reduces lock overhead

→ It maintains the track of what to lock and how to lock.

→ It makes easy to decide, either to lock or data item or unlock a data item

* This type of hierarchy can be graphically represented as a tree.



* The first level or higher levels shows the entire database.

* The second represents a mode of type area.

* The area consists of children nodes which are known as files, no file can be present in more than one area.

* finally, each file contains child nodes known as records, the file has exactly those records that are its child nodes, no record presents in more than one file.

→ Additional Locks :-

* Intention - Shared (IS) :- shared locks

* Intention - Exclusive (IX) :- shared or exclusive

* Shared & Intention - Exclusive (SIX) :- combination of shared & exclusive.

Subject: DBMS

Class Notes

Faculty: R. Akhilesh

Topic: Recovery & Atomicity:

Unit No:

Lecture No:

Link to Session

Planner (SP): S.No. of SP

Book Reference:

Date Conducted:

Page No: 26

* When a system crashes, it may have several transactions being executed and various files opened for them to modify data items.

* Transactions are made of various operations which are atomic in nature.

→ When a DBMS recovers from a crash, it should maintain the following.

* It should check the status of all transactions which were being executed.

* A transaction may be in the middle of some operation, the DBMS must ensure that Atomicity of the transaction in this case.

* It should check whether the transaction can be completed now or it needs to be rolled back.

* No transaction would be allowed to leave the DBMS in an inconsistent state.

→ There are two types of techniques, which can help a DBMS in recovering as well as maintaining the atomicity of transaction.

* Maintaining the log of each transaction and writing them onto some stable storage before actually modifying the database.

* Maintaining shadow paging, where the changes are done on a volatile memory and later

The Actual database is updated.

Log-based Recovery:

Def: Log is a sequence of records, which maintains the Record of Actions performed by a transaction.

* Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.

* If any operation is performed on the database then it will be recorded in the log.

* But the protocol of storing the logs should be done before actual transaction is applied in the database.

→ The Log can be represented as :- ^{for} update
 $\langle T_i, x_j, v_1, v_2 \rangle$ has these fields.

T_i : Transaction identifier

x_j : Data item.

v_1 : old value.

v_2 : New value.

* Let's assume there is a transaction to modify the city of a student. The following logs are written for this transaction.

$\langle T_m, \text{start} \rangle$ // start transaction.

$\langle T_m, \text{city}, 'Noida', 'Bangalore' \rangle \rightarrow$ update

$\langle T_m, \text{commit} \rangle$ // close transaction.

Subject: DBMS

Class Notes

Faculty: R. Akhila

Topic:

Log-based Recovery: Approach to modify database

Unit No:

Lecture No:

Link to Session

Planner (SP): No. of SP

Book Reference:

Date Conducted:

Page No: 27

1. Deferred database modification:

- * This technique occurs, if transaction does not modify the database until it has committed.
- * This method, all the logs are created and stored in the stable storage, and the database is updated when a transaction commits.

2. Immediate database modification:-

- * The immediate modification technique occurs, if the database modification occurs while the transaction is still active.
- * In this technique, the database is modified immediately after every operation, it follows an actual database modification.

Recovery using Log-Record:

→ when the system is crashed, then the system consults the log to find which transaction need to be undone and which need to be redone.

→ If the Log contains Record $\langle T_i, \text{Start} \rangle$ and $\langle T_i, \text{Commit} \rangle$ or $\langle T_i, \text{Commit} \rangle$ then transaction T_i needs to be redone.

→ If the Log contains only $\langle T_m, \text{Start} \rangle$ not contain $\langle T_i, \text{Commit} \rangle$ or $\langle T_i, \text{Aborts} \rangle$ then transaction T_i needs to be undone.

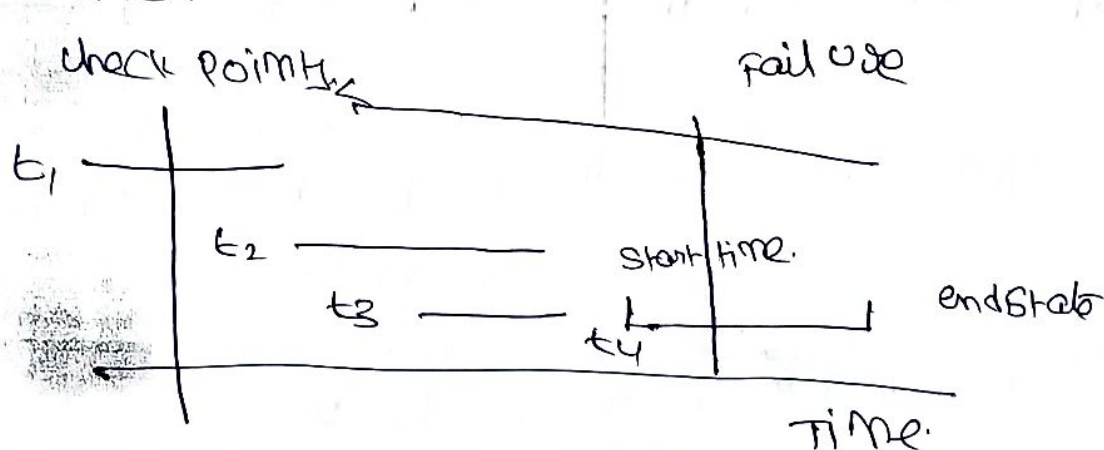
Recovery with concurrent transactions:-

- when more than one transaction are being executed in parallel, the logs are interleaved.
- At the time of recovery, it would become hard for the recovery system to backtrack all logs, and then start recovering.
- To ease this situation DBMS use concept of "check points".

check points:-

- It's keeping and maintaining logs in real time and in real environment may fill out the memory space available in the system.
- Check point is a mechanism where all the previous logs are removed from system and stored permanently in a storage disk.
- It declares a point before which the DBMS was in consistent state and all the transactions were committed.

* When a system with concurrent transaction crashes and recovers, it behaves in following manner.



Subject: DBMS

Faculty:

Topic: Log-based Recovery

Class Notes

Unit No:

Lecture No:

Link to Session

Planner (SP): S No. ... of SP

Book Reference:

Date Conducted:

Page No: 28

* The recovery system reads the log the backwards from the end to last checkpoint.

→ The recovery system reads the log files from end to start it reads the log files from T_4 to T_1

→ It maintains two lists, a Redo-list and Undo-list

* The transaction is put into Redo state, if recovery system sees a log with $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$

→ In the Redo-list and their previous list, all the transactions are removed and then redo before saving their logs.

for eg:- In the log file transaction T_2, T_3 will have $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$

T_1 - have only $\langle T_n, \text{commit} \rangle$

Hence it puts T_1, T_2 , and T_3 Redo-list

* T_4 will have $\langle T_n, \text{start} \rangle$ so T_4 will be put into undo list, since this transaction is not yet complete and failed commit.

Recovery Algorithm :- ARIES

ARIES (Algorithm for Recovery and isolation
Exploiting Semantics)

* Algorithm consists of three steps :-

1. Analyse page
2. Redo page
3. Undo page -

1. Analyse page :- This step identifies the dirty pages in the buffer and the set of transactions Active at the time of crash.

* The Appropriate point in the log where Redo is start is also determining.

2. Redo page :- Necessary redo operation are applied

3. Undo page :- Log is scanned backwards and the operations of transactions Active at the time of crash are undoing in reverse order.